



IUS
INSTITUT
UNIVERSITAIRE
DES SCIENCES

Faculté des Sciences et Technologies (FST)

Rapport du travail de Laboratoire N° 2_Modelisation Mathématiques

Etudiant : Donsam Jean Gabard NOEL

Professeur : Ismael SAINT AMOUR

Niveau : L3

Avril 2026

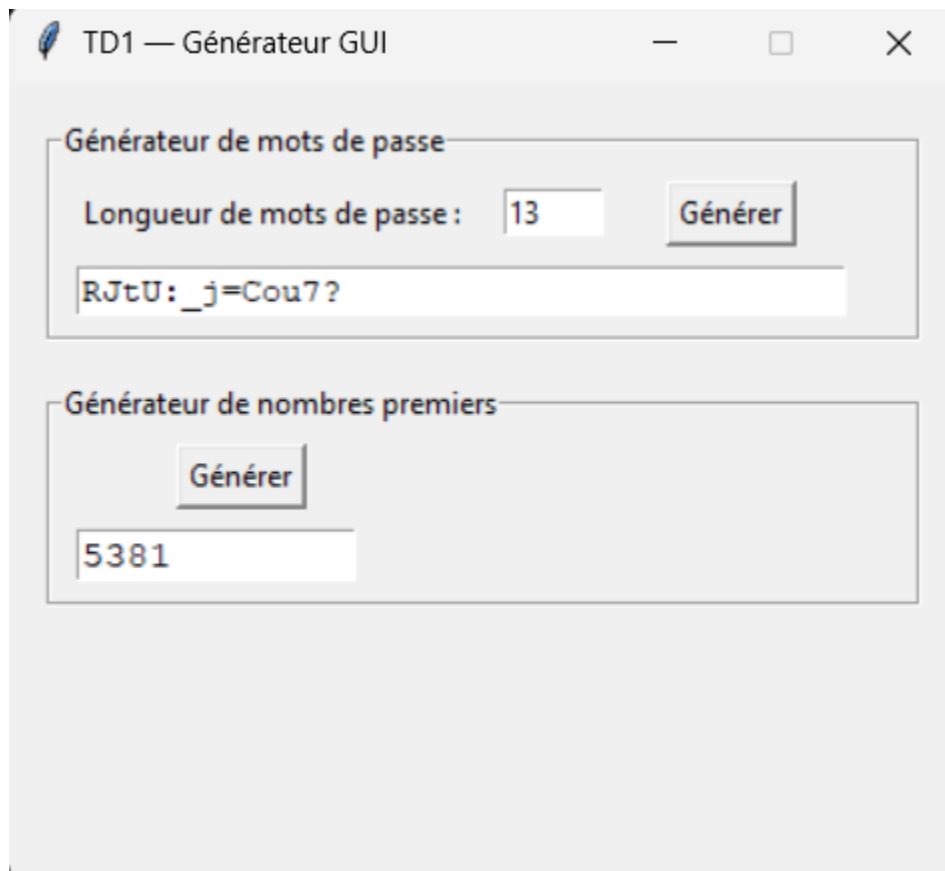
Travaux Dirigés

TD 1: Créer une application GUI qui inclut :

- générateur de mots de passe
- générateur de nombres premiers

```
1  import tkinter as tk
2  import string
3  import random
4  import math
5
6  # ---- Fonctions ----
7
8  def is_prime(n):
9      if n < 2:
10         return False
11         for i in range(2, int(math.sqrt(n)) + 1):
12             if n % i == 0:
13                 return False
14         return True
15
16  def generate_prime(lower=1000, upper=10000):
17      while True:
18         n = random.randint(lower, upper)
19         if n % 2 == 0:
20             n += 1
21         if is_prime(n):
22             return n
23
24  def generate_password():
25      length = int(entry_length.get())
26      characters = string.ascii_letters + string.digits + string.punctuation
27      password = "".join(random.choice(characters) for _ in range(length))
28      entry_password.delete(0, tk.END)
29      entry_password.insert(0, password)
30
31  def generate_prime_gui():
32      p = generate_prime()
33      entry_prime.delete(0, tk.END)
34      entry_prime.insert(0, str(p))
35
36  # ---- Interface ----
37
38  root = tk.Tk()
39  root.title("TD1 - Générateur GUI")
40  root.geometry("380x320")
41  root.resizable(False, False)
42
43  # --- Mots de passe ---
44  frame_pw = tk.LabelFrame(root, text="Générateur de mots de passe", padx=10, pady=8)
45  frame_pw.pack(fill="x", padx=15, pady=(15, 8))
46
47  tk.Label(frame_pw, text="Longueur de mots de passe :").grid(row=0, column=0, sticky="w")
48  entry_length = tk.Entry(frame_pw, width=6)
49  entry_length.insert(0, "16")
50  entry_length.grid(row=0, column=1, sticky="w", padx=5)
51
52  tk.Button(frame_pw, text="Générer", command=generate_password).grid(row=0, column=2, padx=5)
53
54  entry_password = tk.Entry(frame_pw, width=38, font=("Courier", 10))
55  entry_password.grid(row=1, column=0, columnspan=4, pady=(8, 0))
56
57  # --- Nombres premiers ---
58  frame_prime = tk.LabelFrame(root, text="Générateur de nombres premiers", padx=10, pady=8)
59  frame_prime.pack(fill="x", padx=15, pady=8)
60
61  tk.Button(frame_prime, text="Générer", command=generate_prime_gui).grid(row=0, column=1, padx=10)
62
63  entry_prime = tk.Entry(frame_prime, width=12, font=("Courier", 11))
64  entry_prime.grid(row=1, column=0, columnspan=2, pady=(8, 0))
65
66  root.mainloop()
```

- Test de l'application

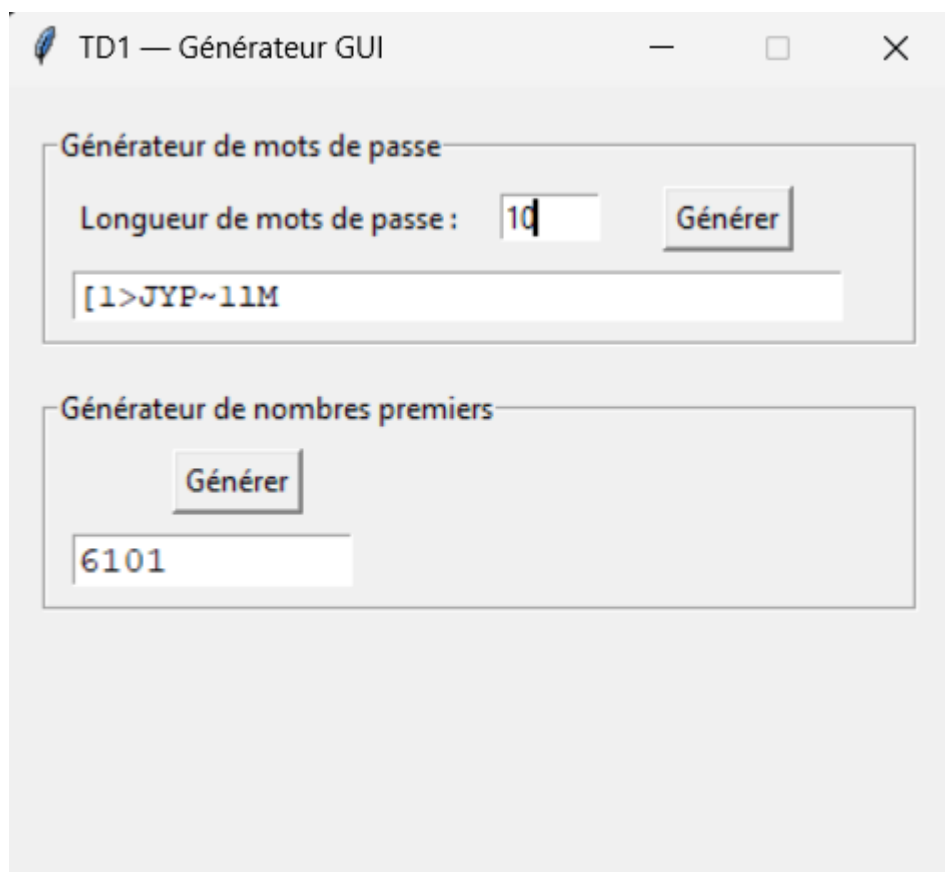


TD1 — Générateur GUI

Générateur de mots de passe

Longueur de mots de passe :

Générateur de nombres premiers



TD1 — Générateur GUI

Générateur de mots de passe

Longueur de mots de passe :

Générateur de nombres premiers

TD 2: Créer un programme Python qui :

- génère des nombres aléatoires
- teste leur primalité retourne un nombre premier

```
2  import random
3  import math
4
5  # --- Fonction de test de primalite ---
6
7  def is_prime(n):
8      """Teste si n est un nombre premier."""
9      if n < 2:
10         return False
11     for i in range(2, int(math.sqrt(n)) + 1):
12         if n % i == 0:
13             return False
14     return True
15
16 # --- Generation avec affichage des essais ---
17
18 def generate_prime_with_trials(lower=100, upper=1000):
19     attempts = 0
20     print("\nRecherche d'un nombre premier entre", lower, "et", upper, "...")
21     print("-" * 45)
22
23     while True:
24         n = random.randint(lower, upper)
25         attempts += 1
26         prime = is_prime(n)
27         statut = "PREMIER" if prime else "compose"
28         print(" Essai", attempts, ":", n, " -> ", statut)
29
30         if prime:
31             print("-" * 45)
32             print("\n Nombre premier trouve :", n)
33             print(" Nombre d'essais      :", attempts)
34             return n
35
36 # --- Programme principal ---
37
38 if __name__ == "__main__":
39     print("-" * 45)
40     print("      TD 2 - Test de primalite")
41     print("-" * 45)
42
43     result = generate_prime_with_trials(lower=100, upper=1000)
44
45     print("\n Verification finale : is_prime(" + str(result) + ") =", is_prime(result))
46     print("-" * 45)
```

▪ Test du Programme

```
=====
|   | TD 2 - Test de primalite
|   |
=====

Recherche d'un nombre premier entre 100 et 1000 ...
-----
Essai 1 : 351 -> compose
Essai 2 : 392 -> compose
Essai 3 : 966 -> compose
Essai 4 : 284 -> compose
Essai 5 : 917 -> compose
Essai 6 : 429 -> compose
Essai 7 : 230 -> compose
Essai 8 : 413 -> compose
Essai 9 : 119 -> compose
Essai 10 : 309 -> compose
Essai 11 : 211 -> PREMIER
-----

Nombre premier trouve : 211
Nombre d'essais      : 11

Verification finale : is_prime(211) = True
=====
```

```
=====
|   | TD 2 - Test de primalite
|   |
=====

Recherche d'un nombre premier entre 100 et 1000 ...
-----
Essai 1 : 450 -> compose
Essai 2 : 906 -> compose
Essai 3 : 946 -> compose
Essai 4 : 651 -> compose
Essai 5 : 847 -> compose
Essai 6 : 607 -> PREMIER
-----

Nombre premier trouve : 607
Nombre d'essais      : 6

Verification finale : is_prime(607) = True
=====
```

```
=====
|   | TD 2 - Test de primalite
|   |
=====

Recherche d'un nombre premier entre 100 et 1000 ...
-----
Essai 1 : 823 -> PREMIER
-----

Nombre premier trouve : 823
Nombre d'essais      : 1

Verification finale : is_prime(823) = True
=====
```

TD 3 :Créer un programme Python qui demande à l'utilisateur de saisir deux nombres et calcule leur PGCD (Plus Grand Commun Diviseur) avec l'algorithme d'Euclide :

```
2  # ---- Algorithme d'Euclide ----
3
4  def pgcd_euclide(a, b):
5      print(f"\nCalcul de PGCD({a}, {b}) :")
6      print("-" * 35)
7
8      etape = 1
9      while b != 0:
10         q = a // b
11         r = a % b
12         print(f"  Étape {etape} : {a} = {b} × {q} + {r}")
13         a, b = b, r
14         etape += 1
15
16     print("-" * 35)
17     print(f"  → PGCD = {a}")
18     return a
19
20 # --- Programme principal ---
21
22 if __name__ == "__main__":
23     print("-" * 35)
24     print("  TD 3 – Calcul du PGCD (Euclide)")
25     print("-" * 35)
26
27     try:
28         a = int(input("\nEntrez le premier nombre (a) : "))
29         b = int(input("Entrez le deuxième nombre (b) : "))
30
31         if a <= 0 or b <= 0:
32             print("Veuillez entrer des entiers strictement positifs.")
33         else:
34             resultat = pgcd_euclide(a, b)
35
36             if resultat == 1:
37                 print(f"\n  {a} et {b} sont premiers entre eux (coprimes).")
38
39             resultat = pgcd_euclide(a, b)
40
41             if resultat == 1:
42                 print(f"\n  {a} et {b} sont premiers entre eux (coprimes).")
43             else:
44                 print(f"\n  Le plus grand diviseur commun de {a} et {b} est {resultat}.")
45
46     except ValueError:
47         print("Entrée invalide. Veuillez saisir des entiers.")
48
49     print("-" * 35)
```

▪ Test du Programme

```
=====
TD 3 – Calcul du PGCD (Euclide)
=====
```

```
Entrez le premier nombre (a) : 97
Entrez le deuxième nombre (b) : 156
```

```
Calcul de PGCD(97, 156) :
```

```
-----
Étape 1 :  $97 = 156 \times 0 + 97$ 
Étape 2 :  $156 = 97 \times 1 + 59$ 
Étape 3 :  $97 = 59 \times 1 + 38$ 
Étape 4 :  $59 = 38 \times 1 + 21$ 
Étape 5 :  $38 = 21 \times 1 + 17$ 
Étape 6 :  $21 = 17 \times 1 + 4$ 
Étape 7 :  $17 = 4 \times 4 + 1$ 
Étape 8 :  $4 = 1 \times 4 + 0$ 
-----
```

```
→ PGCD = 1
```

```
97 et 156 sont premiers entre eux (coprimes).
```

```
=====
TD 3 – Calcul du PGCD (Euclide)
=====
```

```
Entrez le premier nombre (a) : 12
Entrez le deuxième nombre (b) : 56
```

```
Calcul de PGCD(12, 56) :
```

```
-----
Étape 1 :  $12 = 56 \times 0 + 12$ 
Étape 2 :  $56 = 12 \times 4 + 8$ 
Étape 3 :  $12 = 8 \times 1 + 4$ 
Étape 4 :  $8 = 4 \times 2 + 0$ 
-----
```

```
→ PGCD = 4
```

```
Le plus grand diviseur commun de 12 et 56 est 4.
```

